

RELAZIONE TECNICA DI PROGETTO: UninaGym

Insegnamento: Programmazione Object Oriented/Basi Dati Canale 3 (A.A. 2025-26)

Componenti Gruppo: Romano Prisco [DE1000149], Alessandro Papilio [DE1000096], Verazzo Luca [DE1000435]

Referente Gruppo: Romano Prisco [DE1000149]

1. TRACCIA ED ANALISI DEL DOMINIO

Il centro fitness "UninaGym" necessita di un sistema software per coordinare le attività quotidiane. Il sistema deve permettere la registrazione dei **Clienti** e degli **Istruttori**, memorizzando per entrambi i dati anagrafici (id, nome, cognome, email, telefono, data di nascita).

Per ogni cliente, la segreteria deve poter gestire un **Abbonamento** (specificando tipologia, validità e costo mensile). L'accesso fisico alla struttura deve essere monitorato tramite la registrazione di ogni singolo **Accesso** (data/ora e verso), consentendo l'ingresso solo a chi possiede un contratto in corso di validità.

L'offerta formativa si articola in **Corsi** di gruppo (es. Yoga, Crossfit), ciascuno caratterizzato da una capienza massima di partecipanti, un orario prestabilito e un **Istruttore** responsabile. I clienti possono iscriversi ai corsi; tuttavia, il sistema deve gestire le **Iscrizioni** in modo intelligente: se i posti per un corso sono esauriti, il cliente deve essere inserito in una lista d'attesa, pronta a scalare in caso di rinunce.

2. SCHEMA CONCETTUALE E COMPLESSITÀ

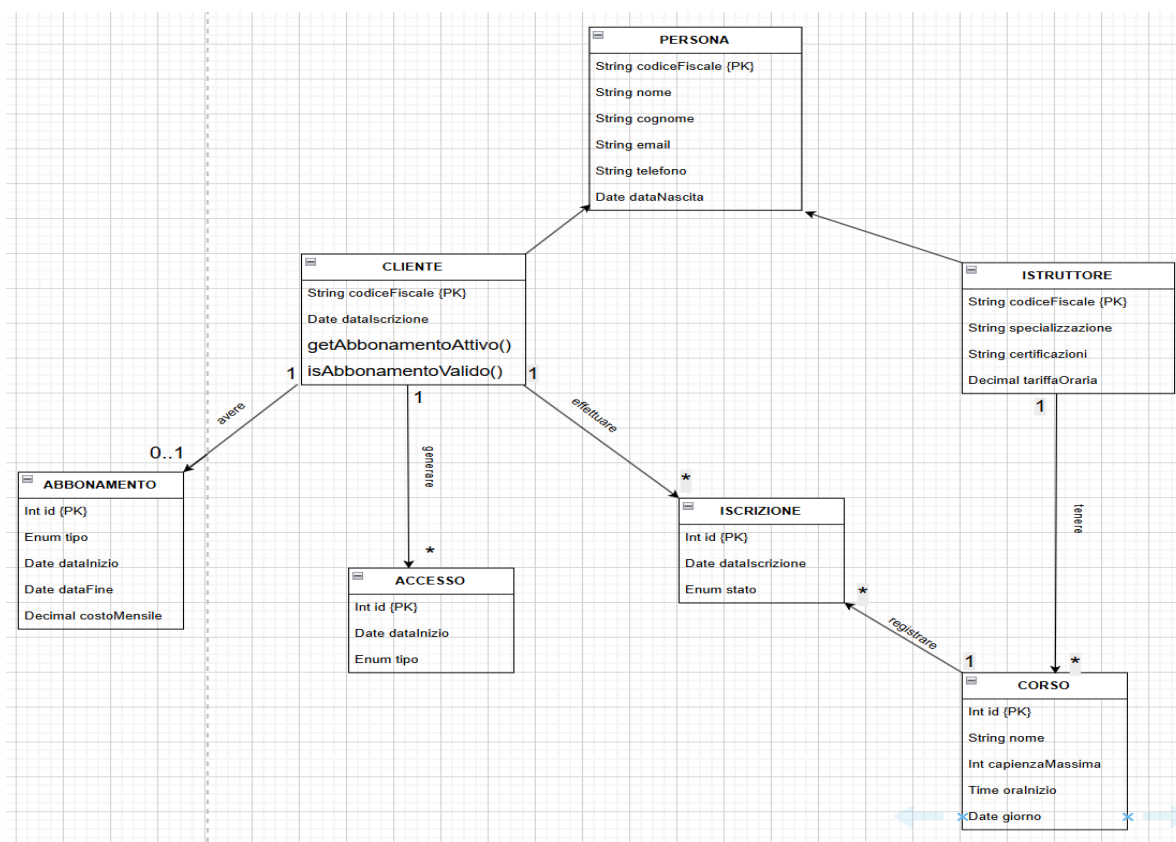
Dall'analisi della traccia sopraesposta, sono state estrapolate le seguenti **7 entità**:

2.1 Catalogo delle Classi (Entity)

1. **Persona:** Superclasse che modella la generalizzazione.
 - o *Attributi:* **String** codiceFiscale {PK}, **String** nome, **String** cognome, **String** email, **String** telefono, **Date** dataNascita.
2. **Cliente (extends Persona):** è una specializzazione della classe Persona.
 - o *Attributi:* **String** codiceFiscale {PK}, **Date** dataIscrizione.
 - o *Metodi:* **getAbbonamentoAttivo()**, **boolean** **isAbbonamentoValido()**.
3. **Istruttore (extends Persona):** è un'altra specializzazione della classe Persona.

- **Attributi:** **String** codiceFiscale {PK}, **String** specializzazione, **String** certificazioni, **Decimal** tariffaOraria.
4. **Abbonamento:** Gestisce il “titolo contrattuale” del cliente.
- **Attributi:** **Int** id {PK}, **Enum** tipo, **Date** dataInizio, **Date** dataFine, **Decimal** costoMensile.
5. **Corso:** Definisce le attività collettive programmabili.
- **Attributi:** **Int** id {PK}, **String** nome, **Int** capienzaMassima, **Time** oraInizio, **Date** giorno
6. **Iscrizione (Classe Associativa):** Risolve la relazione molti-a-molti (N-N) tra Cliente e Corso.
- **Attributi:** **Int** id {PK}, **Date** dataIscrizione, **Enum** stato.
7. **Accesso:** Registro cronologico dei transiti.
- **Attributi:** **Int** id {PK}, **Date** dataAccesso, **Enum** id.

3. ASSOCIAZIONI E CARDINALITÀ



Le relazioni e le cardinalità tra le entità sono state modellate esattamente come segue:

- **Generalizzazione:** Cliente e Istruttore sono sottoclassi della superclasse Persona (Ereditarietà).
- **Cliente 1 — 0..1 Abbonamento:** Un cliente può non avere abbonamenti o averne uno solo attivo.
- **Cliente 1 — * Accesso:** Un cliente può generare molteplici record di accesso.
- **Istruttore 1 — * Corso:** Ogni corso ha un unico istruttore responsabile; un istruttore può tenere più corsi.
- **Cliente * — * Corso (con classe associativa Iscrizione):** Un cliente può iscriversi a più corsi e un corso accoglie n-clienti.

4. REGOLE DI BUSINESS

Il sistema implementa tre regole:

- **T1 - Validazione Temporale:** Impedisce la creazione di un Accesso se la dataFine dell'abbonamento del cliente è passata.
- **T2 - Gestione Overbooking:** All'inserimento di un'Iscrizione, se il numero di record CONFERMATO è pari alla capienzaMassima del corso, lo stato viene forzato a LISTA_ATTESA.
- **T3 - Pricing Dinamico:** Al variare del tipo abbonamento in GOLD, il costoMensile viene ricalcolato automaticamente applicando una riduzione del 30%.

5. ARCHITETTURA SOFTWARE (BCE + DAO)

L'applicazione **UninaGym** è stata progettata seguendo il pattern **BCE (Boundary-Control-Entity)** integrato con lo strato **DAO (Data Access Object)**, garantendo una netta separazione tra l'interfaccia utente, la logica applicativa e la persistenza dei dati. Il codice è organizzato nei seguenti package all'interno del percorso sorgente `it.unina.uninagym`:

- **it.unina.uninagym.model (Entity):** Questo livello contiene le classi di dominio e le relative enumerazioni. Qui viene implementata la logica di incapsulamento per le 7 entità del sistema (Persona, Cliente, Istruttore, Abbonamento, Corso, Iscrizione, Accesso) e la gerarchia di ereditarietà tra la classe astratta Persona e le sue specializzazioni.
- **it.unina.uninagym.dao (Data Access Object):** Comprende le interfacce e le relative implementazioni concrete (`dao.impl`) che utilizzano il driver JDBC per interagire con il database PostgreSQL.

- **it.unina.uninagym.controller (Control):** Rappresenta il “cuore logico” del sistema. I controller coordinano il flusso di dati tra la GUI e i DAO, validando l'input e applicando le regole di business prima della persistenza.
- **it.unina.uninagym.gui (Boundary):** Gestisce l'interfaccia grafica basata su libreria Swing. Le classi in questo package si occupano esclusivamente della presentazione dei dati e della cattura degli eventi utente, delegando ogni operazione di calcolo o modifica ai rispettivi controller.
- **it.unina.uninagym.db:** Package dedicato alla gestione della connessione JDBC, garantendo un'unica istanza di collegamento al database durante l'esecuzione del software.

6. REPOSITORY E DIAGRAMMA UML

- **GitHub:**
https://github.com/RomanoPrisco/Progetto_POO.git

